

# 伟大的复习资料

操作系统：

系统软件

它是**资源管理平台**，**运行程序的平台**，也是**为用户提供服务的平台**

**引导程序的工作内容包括**

1. 初始化设备
2. 把操作系统载入到内存中
3. 运行**第一个进程 init()**, 等待事件发生

操作系统**服务的对象**：用户、程序

**系统调用**：封装 os封装了物理机器指令 **昂贵、慢、依赖于系统**

**API**：封装 系统调用/机器指令 **简单、可迁移**

程序通过应用程序接口（API）访问，而不是直接使用系统调用。

向系统调用**传参方式**：

- 寄存器
- 块
- 栈

系统程序**不属于内核**，但属于**操作系统的一部分**

应用程序vs系统程序：

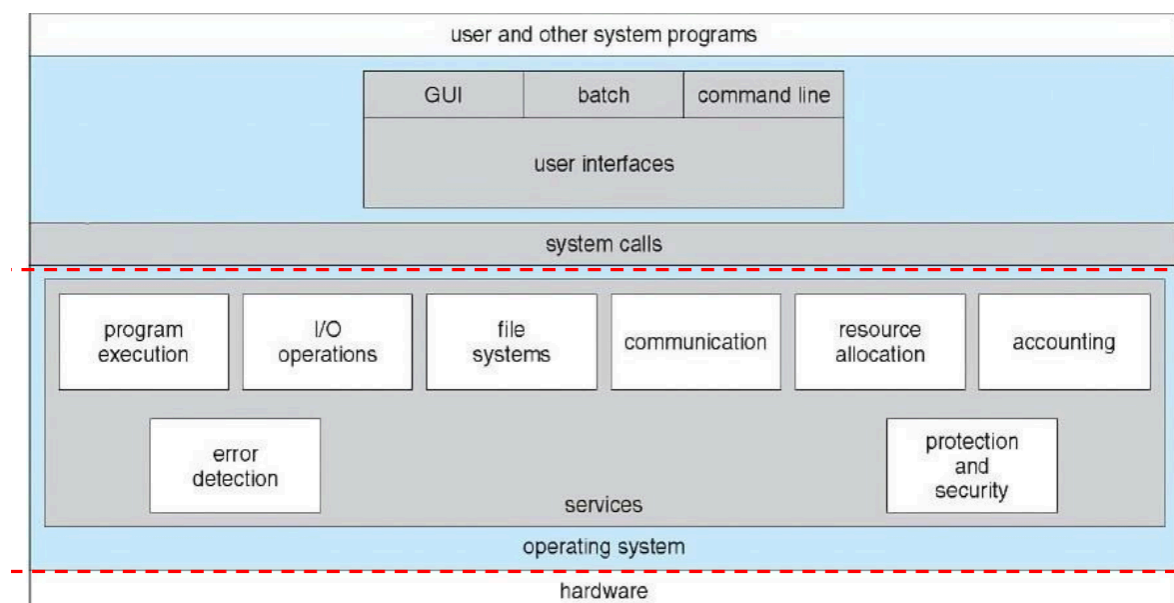
- 应用程序：**直接为用户服务**
- 系统程序：为**用户和其他的程序**提供**软件和软件平台**（platform）

策略(Policy): 做什么

机制(Mechanism): 怎么做

结构：分层

内核：**接口和驱动程序，硬件之上和系统调用接口下的所有部分**，包括文件系统、CPU调度、内存管理和其他操作系统功能



内核结构：五种

- 简单结构
- 微内核
- 层次结构
- 模块结构：面向对象（现在大部分采用）
- 混合结构

微内核的优缺点：

- 优点：便于扩充微内核、便于移植操作系统到新架构系统上、更稳定（更少的代码运行在核心态）、更安全
- 缺点：用户空间和内核空间通信的系统开销增加

虚拟机：

通过软件模拟的具有完整硬件系统功能的、运行在一个完全隔离 环境中的完整计算机系统

资源被**共享**、虚拟机**隔离**（提供对系统资源的完全保护）

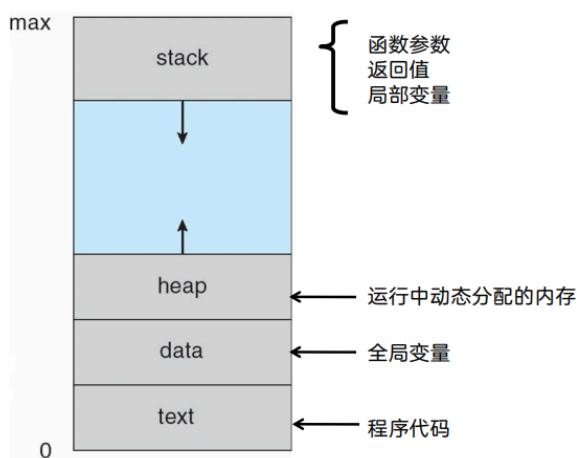
## 进程：进行中的程序

任务vs作业：**作业是任务的实例**

当一个**程序被载入内存**时，这个程序就会变成进程

### 进程的表现形式：

#### 进程在内存中的表现形式

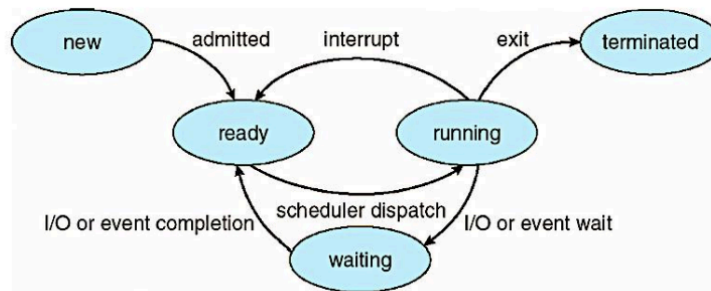


1. **代码段** (code section)：又称文本段 (text section)，通过程序计数器和处理器寄存器的内容来表示当前活动。
2. **数据段** (data section)：包含全局变量。
3. **栈** (stack)：包含临时数据，如函数参数，返回地址，局部变量。
4. **堆** (heap)：在进程运行期间动态分配的内存。

每个进程在操作系统内用**进程控制块 (Process Control Block: PCB)** 来表示，又称任务控制块 (Task Control Block: TCB)，即**进程在操作系统中体现的数据结构**

**init ()**：此进程pid为1

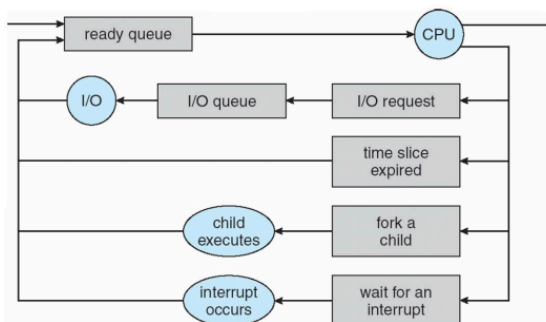
### 3、进程状态



进程在运行时，自身的状态会发生变化

1) 新的(new): 进程正在被创建； 2) 就绪(ready): 进程等待分配处理器； 3) 运行(running): 指令正在被执行； 4) 等待(阻塞)waiting(blocking): 进程等待某个事件的发生； 5) 终止(terminated): 进程完成执行。

操作系统持有就绪队列和一组设备队列，进程在多个调度队列之间移动



进程运行（运行状态）过程中，可能发生下面事件中的一种并进入到就绪状态：

1. 进程可能发出一个 I/O 请求，并被放到 I/O 队列。
2. 进程可能创建一个新的子进程，并等待该子进程结束。
3. 进程可能会由于等待中断而强制释放 CPU，并被放回就绪队列。
4. 用完时间片（time slice / time quantum），并被放到就绪队列。

调度程序分为：

长期、短期、中期（P16）

#### 上下文切换

涉及到PCB

上下文切换时间是额外的开销，因为上下文切换时系统不能做什么有用的工作

为了提高上下文切换速度，有的处理器（如 Sun UltraSPARC）提供多组寄存器集合，上下文切换只需简单地改变当前寄存器组的指针

## 进程创建4个步骤:

- SETP 1: 在系统内部创建进程控制块
- SETP 2: 分配内存
- SETP 3: 载入可执行文件 (通过调用 exec()系统调用)
- SETP 4: 初始化程序

## For Example:

- fork() 系统调用创建新的进程。
  1. 复制父进程PCB
  2. 分配内存空间
- 调用 exec() , 以便重载运行程序。
  3. 从磁盘载入二进制程序
  4. 初始化

级联终止: 父进程停子进程也停

僵尸进程产生原因: 父进程没有等待

进程通信:

- 消息传递: 需要内核的干涉
- 共享内存: 以最快的速度进行方便的通信, **需要同步机制**

消息队列的实现方式有以下三种: 零容量、有限容量、无限容量

远程过程调用-RPC: Remote Procedure Calls

远程方法调用-RMI: Remote Method Invocation (in Ja

每个独立的远程过程都有一个**存根**

管道是**半双工**的, 数据只能**单向通信**

双方通信时, 需要建立起两个管道

## 线程

定义: 大部分现代应用软件都是多线程, 线程是**运行在应用进程里的**、**比进程更小的运行单位**

创建**进程**和创建**线程**: **进程重量级, 代价高; 线程轻量级;**

相对创建进程，创建线程比较简单，只需要复制**栈**和**寄存器**的内容

**一个进程可以拥有多个线程，在这些线程之间，共享：**

- 代码段、全局变量
- （打开的文件的）文件标识符
- 工作环境（当前目录、用户权限等）

**总结：每个线程是独立的调度对象，一个进程可以拥有单个或多个线程并共享内存空间**

**多线程优点：**

- 响应速度快
- 资源共享
- 经济（对应上文说线程切换比进程切换更经济）
- 多处理器体系结构的利用（多线程可以在多处理器上**并行**）

并发vs并行：

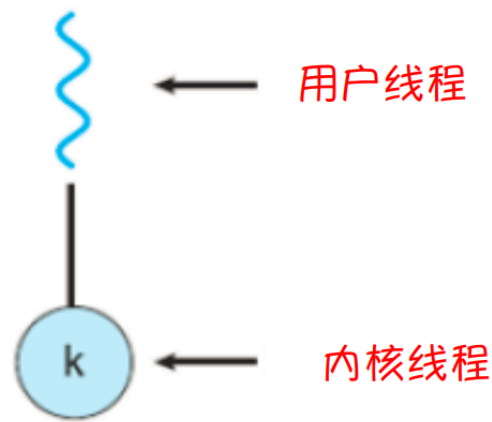
并发：实际上是串行的，为多任务分配时间片（time slice）

并行：真正意义上的同时运行，需要多处理器

**线程分为：用户线程、内核线程，二者缺一不可**

- 用户线程受内核的支持；用户线程是由用户层的线程库来管理，无需内核管理
- 内核线程是由内核来管理；由内核来进行维护和调度

两者关系朴素理解：我们能感知到的线程，比如java里面写代码创建线程，这个就是用户线程。那么这个java多线程相当于是API嘛，那么java会调用操作系统提供的**系统调用**来实现这个用户线程。那么操作系统真正管理的是内核线程，内核线程负责实际处理“线程”。那么操作系统可以让一个内核线程负责一个用户线程的工作，也可以一个内核线程负责多个用户线程的工作，也可以多个内核线程负责一个用户线程的工作等等。总之这些取决于操作系统实现该任务的**策略**。用户线程相当于是一种逻辑抽象。



- 用户线程和内核线程之间必然会存在的一种关系。有一对一、多对一、多对多关系，建立起来的这种关系我们称为**映射**。
- 讨论它们之间关系之前，我们先做以下假设
  1. 操作系统支持内核线程
  2. 把内核线程看成是一个进程
  3. 用户线程基于内核线程运行，即用户线程阻塞内核阻塞、内核线程阻塞用户线程也阻塞。

**\*被阻塞的线程会进入到等待状态**

所以有如下几种模型（映射关系）：

- 一对一

一个用户线程映射到一个内核线程

**优点：**1.提供**并发**功能，可以在多cpu上并行 2.一个线程**阻塞**系统调用时，可以**调度另一个线程继续执行**

**唯一的缺点：**创建一个用户线程就要创建一个内核线程，内核**负担重**

- 多对一

多个用户线程映射到一个内核线程，线程由用户空间管理

**缺点：**一个用户线程被阻塞，则其他用户线程也被阻塞（因为是同一个内核线程负责处理这些用户线程）**无法提供并发功能**

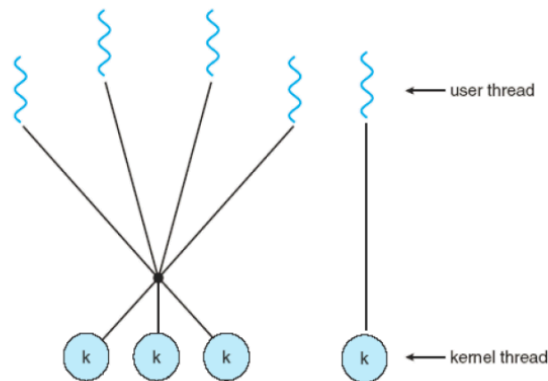
- 多对多

### 没有上述两种模型的缺点

当一个线程执行**阻塞**系统调用时，内核**能调度另一个线程**的执行，但需要**提供调度机制**

- 二级模型（混合模型）

- 多对多模型的变种，即允许用户线程绑定一个特定的内核线程
- Examples: IRIX, HP-UX, Tru64 UNIX, Solaris 8 and earlier



线程库：为程序员提供的、创建和管理线程的 API，主要有以下两种方法来 实现

- 非嵌入到内核的方式：是在用户空间中提供一个没有内核支持的库，此库的所有代码和 数据结构都存在于用户空间中
- 嵌入到内核的方式 是操作系统直接支持的内核库，此时所有代码和数据结构都存在于内核

线程库函数：

重要，可能会考读代码题，如下



```
#include <pthread.h>
```

```
int pthread_create( pthread_t *tidp, //thread ID point  
const pthread_attr_t *attr, //set attribute  
(void*)(*start_rtn)(void*), //thread  
void *arg); // thread parameter
```

- 创建线程系统调用。
- 若线程创建成功，则返回0。若线程创建失败，则返回出错编号

```
void pthread_exit()
```

- 终止线程执行

```
#include <pthread.h>
```

```
int pthread_join( pthread_t thread, //thread ID  
void ** retval); // store thread return value
```

- 等待一个线程结束
- 成功则返回零，否则返回错误号

```
int pthread_attr_init( pthread_attr_t * attr );
```

- 初始化线程对象属性
- 成功则返回零，否则返回错误号

```

hbpark@hbpark-VirtualBox: ~/src/thread
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum;
void * runner01(void * param); // thread 1
void * runner02(void * param); // thread 2

int main (int argc, char * argv[])
{
    sum = 0;
    pthread_t tid01;
    pthread_t tid02;
    pthread_attr_t attr;
    if (argc != 2) {
        fprintf(stderr, "usage: a.out <integer value>\n");
        return -1;
    }
    if (atoi(argv[1]) <= 0){
        fprintf(stderr, "%d must be > 0\n", atoi(argv[1]));
        return -1;
    }
    pthread_attr_init(&attr); // get the default attributes
    pthread_create(&tid01, &attr, runner01, argv[1]);
    pthread_create(&tid02, &attr, runner02, argv[1]);
    pthread_join(tid01, NULL);
    pthread_join(tid02, NULL);

    pthread_exit(0);
}

```

```

void * runner01(void * param)
{
    int i, upper = atoi(param);
    for ( i = 1; i <= upper; i++)
        sum += i;
    printf("[thread 1] sum = %d\n", sum);
}

void * runner02(void * param)
{
    int i, lower = atoi(param);
    for (i = 1; i <= lower; i++)
        sum -= i;
    printf("[thread 2] sum = %d\n", sum);
}

```

```
hbpark@hbpark-VirtualBox: ~/src/thread
hbpark@hbpark-VirtualBox:~/src/thread$ ls
Files  read  thread.c  thread-signal.c
hbpark@hbpark-VirtualBox:~/src/thread$ ./thread 3
[thread 2] sum = 0
[thread 1] sum = 6
hbpark@hbpark-VirtualBox:~/src/thread$ ./thread 3
[thread 2] sum = -6
[thread 1] sum = 0
hbpark@hbpark-VirtualBox:~/src/thread$ ./thread 3
[thread 1] sum = 6
[thread 2] sum = 0
hbpark@hbpark-VirtualBox:~/src/thread$
```

进程中的某一线程调用fork () , 什么时候会复制所有线程, 什么时候只复制调用fork的线程?

答: 如果调用exec () , 只复制一个, 如果不调用exec () , 复制全部

线程池:

- 主要思想是在进程开始时创建一定数量的线程, 并放入到进程池中等待工作
- 优点:
  - 用现有线程处理请求要比等待创建新的线程要快
  - 限制了在任何时候可用线程的数量, 这可以防止大量创建并发线程, 有助于管理

## CPU调度

多道程序的目的: 使CPU使用率最大化

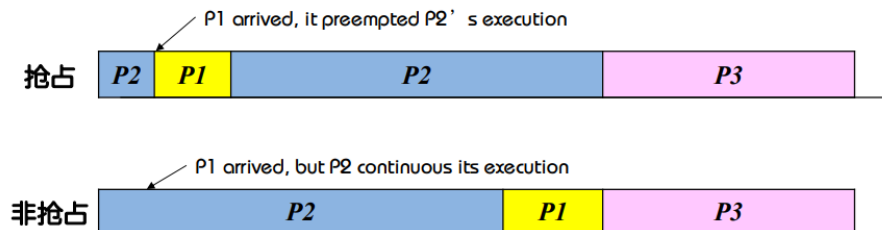
一个进程的执行由CPU执行 (CPU区间) 和IO等待 (IO区间) 组成

一般而言: IO为主的程序短CPU区间多; CPU为主的程序长CPU区间多

CPU调度:

- 每当CPU空闲时, 调度程序(短期调度程序)从就绪队列中选择一个进程, 并为之分配CPU;
  - 可以采用各种不同的排序方式对就绪队列中的进程进行排序(可以理解为不同的调度算法)

- **非抢占调度 (nonpreemptive)** 1,4
  - 一旦把CPU分配给一个进程，直到该进程结束之前，不能把CPU分配给其他进程
- **抢占调度 (preemptive)** 2,3
  - 进程在执行过程中，可以被其他进程抢占CPU使用权



- 分派程序是一个模块，用来将CPU的控制交给由短期调度程序选择的进程。其功能包括：
  1. 切换上下文 (context switch)
  2. 切换到用户模式
  3. 跳转到用户程序的合适位置，以重新启动程序

合适的位置：The address space which is specified by the program counter (PC)
- 分派延迟 (Dispatch Latency)
 

分派程序停止一个进程而启动另一个进程所要花的时间称为分派延迟

### CPU调度准则：

[Scheduling Criteria & Performance Metrics](#)

不同的调度算法具有不同的属性，分析CPU调度算法需要考虑以下准则

1. **CPU使用率 (CPU Utilization)**：需要使CPU尽可能忙
2. **吞吐量 (Throughput)**：指的是在一个时间单元内所完成的进程数量
3. **周转时间 (Turnaround Time)**：从进程提交到进程完成的时间段称为周转时间。周转时间为所有时间段之和，包括等待进入内存，在就绪队列中等待，在CPU上执行和I/O执行
4. **等待时间 (Waiting Time)**：在就绪队列中等待的时间
5. **响应时间 (Response Time)**：从提交请求到产生第一相应的时间。注意，它是开始响应所需要的时间，而不是输出响应所需要的时间（交互系统）

### 调度算法：

很重要，请每一种都去看第五章PPT的例子。这边只列举一些可能出道填空的知识点

**护航效果**(convoy effect): 所有其他小进程都等待一个大进程释放CPU，即 **短进程跟在长进程后面**

大进程指CPU区间长，小进程反之

- 优先级调度的问题：**饥饿（又叫无限阻塞）**，即低优先级进程无法运行（因为永远是高优先级进程在运行，轮不到低优先级）
- 解决方法：**老化**，即随着等待时间增加，增加进程的优先级

**老化**可以使用**多级反馈队列调度**实现

多处理器调度：

- 对称多处理：每个处理器各自执行调度，**不允许处理器间移动**
- 非对称多处理：只有一个处理器进行调度任务，允许处理器间移动

**处理器亲和性**：尽量把进程放入到同一个处理器上运行

## 进程同步

（类似数据库的锁那一块）

确保操作共享数据的代码段的执行同步(互斥运行)，不能让多个进程同时运行操作共享数据的代码段

竞争条件问题：不好描述，PPT上是AB两个进程同时在一个账户上取钱的例子

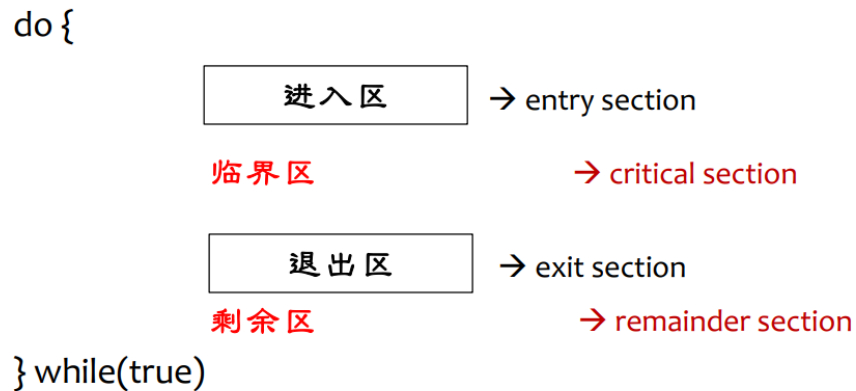
**临界区**：

- 多个进程同时操作共享数据时，每个进程拥有操作共享数据的代码段（程序段），而这代码段称为临界区

（说人话就是涉及到共享数据的代码片段叫临界区）

解决竞争条件问题的关键：**单执行，可进入**

- 确保**单个进程**在临界区**执行**
- 确保**其他进程可以进入**临界区



### 解决临界区问题需要满足的条件：三个

- **互斥**  
| 某一进程进入临界区，其他进程就不进入
- **前进**  
| 如果没有进程在临界区执行，确保一个进程进入临界区
- **有限等待**  
| 一个进程从请求进入临界区到被允许，必须有限时间等待

### Peterson算法

关键：两个变量

- turn 表示哪个进程**可以**进入临界区
- flag【2】 表示哪个进程想要进入临界区（**请求进入**）。值为true表示该进程可以进入临界区

一般形式：

```

do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
        critical section
    flag[i] = false;
        remainder section
} while (true);

```

Giving way to the other process

举例：

```

Initialization
flag[2] = {false; false};
turn = 0;

```

```

PROCESS 0
do{
    flag[0] = true;
    turn = 1;
    while(flag[1] && turn == 1);
        //critical section
    flag[0] = false;
} while(true)

```

```

PROCESS 1
do{
    flag[1] = true;
    turn = 0;
    while(flag[0] && turn == 0);
        //critical section
    flag[1] = false;
} while(true)

```

要学会默写上面这段代码，这段代码表示解决了临界区问题的两个进程的代码。其中critical section表示临界区，不用具体给出

不可被中断的指令：原子指令

**Peterson's 算法、TestAndSet( ) 和 swap( ) 原子指令**  
**(atomic instruction)操作会存在忙等待的问题 (busy waiting)**

声明一个布尔变量 `waiting` 确保有限的等待

1. 局部变量 `key`，初始化为 `true`；每个进程
2. 全局变量 `lock`，初始化为 `false`；
3. 用变量 `waiting[i]` 表示等待进入临界区的进程，初始化为 `false`；

可以进入临界区的 Condition：

当 `lock` 或 `waiting[i]` 为 `false` 时

```
do {
    waiting[i] = true;
    key = true;
    ① while(waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = false;
    // critical section
    j = (i + 1) % n;
    ② while((j != i) && !waiting[j])
        j = (j + 1) % n;
    ③ if (j == i) // entered
        lock = false;
    ④ else // waiting
        waiting[j] = false;
    // remainder section
} while (true);
```

- `i` 表示当前运行的进程
- `n` 表示进程的数量
- `j` 表示 `i` 进程下一个进程

检查下个进程 `j` 是否要进入临界区

1. 要是 `waiting[j] == false`，即不想进入临界区或已经进入临界区，那么跳过，检查在下一个进程； ②
2. 要是 `waiting[j] == true`，即想进入临界区，就允许进入，并把 `waiting[j]` 设置成 `false`； ④
3. 检查了一遍，没有进程想进入临界区，就把 `lock` 设置成 `false`；

同样，上面这段尽量理解并记忆。可以看PPT的例子理解。类似于数据库的加锁

**信号量：无忙等待的同步工具（前面 `peterson` 和 `testandset` 都是同步工具）**

- 适用于单个或多个资源的同步操作
- **二进制信号量又称互斥锁**



## 1. 二进制信号量(binary semaphore), 又称互斥锁 (mutex lock)

- 适用于单资源的共享, mutex值为资源数量, 初始化为1
- 信号量的值只能为0或1

```
do{
    waiting(mutex);
    //critical section
    signal(mutex);
    //remainder section
} while (TRUE);
```

```
do{
    waiting(mutex);
    //critical section
    signal(mutex);
    //remainder section
} while (TRUE);
```

## • 计数信号量:

### 2. 计数信号量(counting semaphore)

- 适用于多资源共享, 共享资源的数量为  $n$
- $wait(n)$  操作为减,  $signal(n)$  操作为加, 当  $n$  为0时表明所有资源都被占用

**Tips:** 可以适用于优先约束 (precedence constraint) 例子, 假设要求  $P1$  的语句  $S1$  完成之后, 执行  $P2$  的语句  $S2$ , 共享信号量  $synch$ , 并初始化为0

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```

Mutex lock  $\approx$  binary semaphore  
But the process that locks the mutex  
must be the one to unlock it.

```
P1:
do{
    wait (mutex);
    //critical section
    signal(mutex);
    //remainder section
} while (TRUE);
```

```
P2:
do{
    wait (mutex);
    //critical section
    signal(mutex);
    //remainder section
} while (TRUE);
```

## 多资源共享

假设共享同类的两个资源 ( $n=2$ )

- 信号量 *mutex* 初始化为 2
- *P1, P2, P3* 竞争

```
P3:
do{
    wait (mutex);
    //critical section
    signal(mutex);
    //remainder section
} while (TRUE);
```

## 优先约束

- 要求 *P1* 运行后, *P2* 运行
- 信号量 *mutex* 初始化为 0

```
P1:
do{
    //write operation
    signal (mutex);
} while (TRUE);
```

```
P2:
do{
    wait (mutex);
    //read operation
} while (TRUE);
```

上面一个“多资源共享”和一个“优先约束”需要会默写

信号量的实现关键是保障 `wait()` 和 `signal()` 操作的原子执行, 即必须保障没有两个进程能同时对同一信号量执行 `wait()` 和 `signal()` 操作

上述信号量存在问题: **忙等待、自旋锁**

解决: 改进信号量数据结构 (我们没要求会)

**死锁:** 两个或多个进程无限地等待一个事件, 而该事件 只能由这些等待进程之一来产生

**饥饿**：无限期的等待，即进程在信号量内无限期的等待

### 经典同步问题：

- 读写问题

#### 写者进程

第 6 章 进程同步

66

1. 信号量 `wrt` 为读者和写者进程共享，初始化为 1，从而达到写操作互斥的目的
2. 变量 `readcount` 用来跟踪有多少进程正在读对象，初始化为 0
3. 信号量 `mutex` 用于确保在更新变量 `readcount` 的互斥，初始化为 1

```
while (true){  
    wait(wrt);  
    // writing is performed  
    signal(wrt);  
}
```

#### 读者进程

第 6 章 进程同步

67

```
while (true)  
    wait(mutex);  
    readcount ++;  
    if (readcount == 1) —————> 第一位读者加锁  
        wait(wrt);  
    signal(mutex)  
    // reading is performed
```

```
    wait(mutex);  
    readcount --;  
    if (readcount == 0) —————> 最后一位读者解锁  
        signal(wrt);  
    signal(mutex);  
}
```

- 哲学家进餐

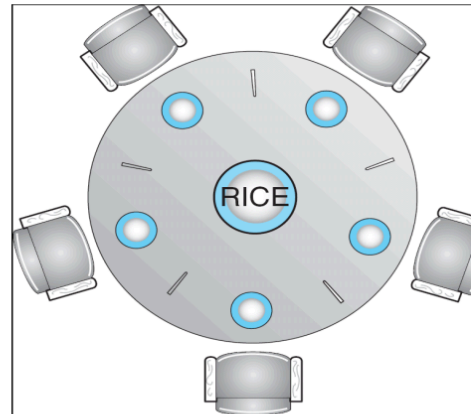
1. 哲学家们用一生来思考和吃饭
2. 一碗米饭
3. 5只筷子
4. 同时有两只筷子才能吃饭

是多个进程共享多个资源的问题

共享数据

```
semaphore chopstick[5];
```

```
// Initially all values are 1
```



```
do {
    wait(chopstick[i]) //左手边筷子
    wait(chopstick[(i+1) % 5]) //右手边筷子
    ...
    // eat
    ...
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    ...
    // think
    ...
} while (true);
```

What is the problem of this algorithm?

问题：可能死锁

解决：

制定如下规定

1. 区分哲学家所处的三个状态

THINKING, HUNGRY, EATING

2. 哲学家  $i$  只有在其两个邻居不进餐时，才能拿起筷子进餐

$(state[(i+4)\%5] \neq EATING) \text{ and } (state[(i+1)\%5] \neq EATING)$

3. 声明条件变量-哲学家

*condition self[5], 提供wait() 和 signal()操作*

```
monitor DP {
    enum { THINKING; HUNGRY, EATING } state [5];
    condition self [5];
    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }
    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5); // left
        test((i + 1) % 5); // right
    }
}
```

```
void test (int i) {
    if ( (state[(i + 4) % 5] != EATING) && (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) )
    {
        state[i] = EATING ;
        self[i].signal();
    }
}
initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}
```

test函数表示测试i能不能进餐，判断了i的左右邻居是否在进餐，同时判断i本身是否hungry（需要进餐）。如果测试能进餐，test函数同时会让i开始进餐。

上述图片代码均需要理解记忆。可能出代码填空。

## 死锁

### 死锁的产生：

- 进程占用一个资源
- 并请求被其他进程所占用的资源

### 死锁特征：

- 互斥
- 占用并等待
- 非抢占
- 循环等待

### 资源分配图：

1. 节点集合  $V$  分为 (1) 进程集合  $P$  和 (2) 资源集合  $R$

- 进程节点集合：  $P = \{P_1, P_2, \dots, P_n\}$
- 资源节点集合：  $R = \{R_1, R_2, \dots, R_m\}$

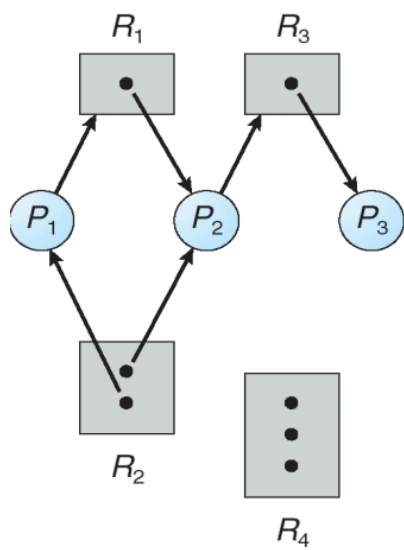
2. 边集合  $E$

- $P_i \rightarrow R_j$

表示进程  $P_i$  已经申请使用资源类型  $R_j$  的一个实例

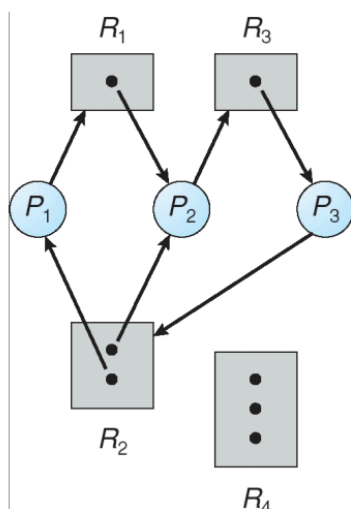
- $R_j \rightarrow P_i$

表示资源类型  $R_j$  的一个实例已经分配给进程  $P_i$



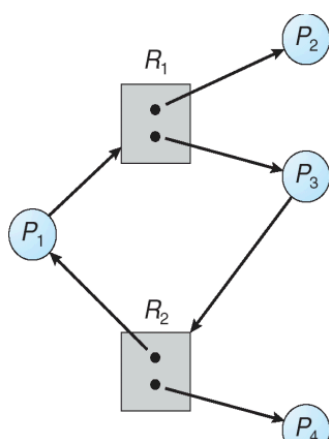
- $R_1 \rightarrow P_2$
- $R_2 \rightarrow P_1, R_2 \rightarrow P_2$
- $R_3 \rightarrow P_3$
- $P_1 \rightarrow R_1$
- $P_2 \rightarrow R_3$

会考让你如何打破死锁，如下图：



由于 $P_3$  申请资源  $R_2$ ，导致资源的请求形成了环。

Q:那么，形成环就会发生死锁吗？



$P_2$  或  $P_4$  释放资源，可以打破环

### 结论：

- 如果分配图没有环，就没有死锁
- 如果分配图有环，就有可能发生死锁

如果每个资源类型只有一个实例，就肯定会发生死锁

如果每个资源类型有多个实例，就有可能处于死

### 预防死锁

从死锁产生的条件入手，对应：（理解即可）



### (1) 互斥-通常不能通过否定互斥条件来预防死锁

- 不可共享的资源必须要确保互斥，如打印机
- 可共享的资源不要求互斥访问，如只读文件

### (2) 占有并等待

确保当一个进程请求一个资源时，它不能占有其他资源。  
实现的方法如下

A. 每个进程在执行前申请并获得所有资源

B. 进程只有在不占用资源时, 允许进程申请资源

### (3) 非抢占

为了确保这一条件不成立，可以是使用如下协议：

如果一个进程占有资源并申请另一个不能分配的资源，那么其现已分配的资源可被抢占

如果一个进程申请一些资源，那么首先检查它们是否可用

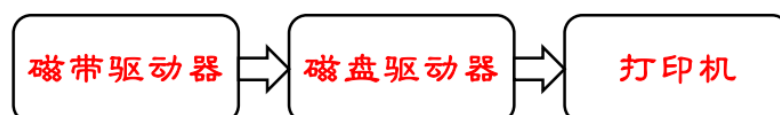
- a) 如果可用，就分配
- b) 如果不可用，检查这些资源是否已分配给其他等待额外资源的进程，如果是，那么就抢占；
- c) 如果不可用，且也没有被其他等待额外资源的进程占有，那么就等待

### (4) 循环等待

- 为每个资源类型分配一个唯一的整数
- 每个进程按递增顺序申请资源

例子

- 磁带驱动器：1，打印机：12，磁盘驱动器：5



银行家算法：

两个部分，分清哪个是哪个

具体见PPT，只给出需要记忆部分

### 1.资源请求算法

每次进程请求资源的时候，运行资源请求检测算法，确认是否允许请求。

#### Resource Request Algorithm:

```
IF ( Request i <= Need i) //检测资源的请求是否合法
  IF (Request i <= Available i){ //检测可用资源能否满足请求
    Available i = Available i - Request i ;
    Allocation i = Allocation i + Request i ;
    Need i = Need i - Request i ;
    do Safety Check Algorithm; //检测分配资源后是否安全
  }
ELSE
  waiting;
End IF
ELSE
  error message;
End IF
```

### 2.安全检查算法

#### Safety Check Algorithm:

```
Work = Available;
For all i, Finish[i] = false;
For all i do
  IF ( Finish[i] == false && Need i <= Work)
    Work = Work + Allocation i
    Finish[ i ] = true
  End IF
End For
IF for all i, Finish[i] == true
  Then the system is safety
End IF
```

PPT上还有对应算法运行时的每一步分析，看一遍理解即可。

死锁检测算法:

考画图, 见PPT那道题, 会原题即可

内存

GCC 编译阶段和命令:

## 1. Preprocessing

```
$ gcc -E -o hello.i hello.c
```

```
$ cpp hello.c hello.i
```

## 2. Compile

```
$ gcc -S -o hello.s hello.c
```

```
$ cc -S hello.i
```

## 3. Assembling

```
$ gcc -c -o hello.o hello.c
```

```
$ as -o hello.o hello.s
```

## 4. Linking

```
$ gcc -o hello hello.c
```

```
$ ld hello.o ( see the next slide )
```

预处理、编译、汇编、连接

将hello.c 编译为可执行文件的命令: (一行) gcc -o hello hello.c

-o hello 表示输出文件名为hello

gcc命令为: gcc -o 【输出可执行文件名】 【.c文件名】

链接: 分为静态、动态

- **静态链接**：加载程序将库的内容合并到二进制程序镜像中，一直驻留在内存
  - **动态链接**，链接延迟到运行时
    1. 有个小程序，叫**存根(stub)**，用来定位适当的内存驻留库程序，或如果该程序不在内存时应如何装入库
    2. 存根首先检查所需子程序是否在内存中，如果不在，就将子程序装入内存
    3. 存根会用子程序地址来替换自己，并开始子程序
    4. **动态链接通常适用于系统库，如语言库，由操作系统管理**
- 

二者对比：

**动态链接和动态加载都是需要时加载到内存。但，**

- **动态链接**

程序启动时建立了链接（即只有链接地址），需要时载入内存，由操作系统决定

- **动态加载**

通过程序的方法来控制加载，由程序员决定

内存分配方式：

连续分配：**每个进程位于连续的内存区域**（进程就是程序加载进内存并运行）

- 多分区：多个固定大小分区
- 可变分区：孔

2. **可变分区方法**：整个内存空间是就是一个大孔, 操作系统用表来记录已用和未用内存,

- 孔, 每当新的进程载入时, 分配足够大(?) 的孔
- 系统需要用表来记录已分配孔和未分配孔的信息

- I. **首次适应(first fit)**: 分配足够运行进程的第一个找到的孔
- II. **最佳适应(best fit)**: 分配足够运行进程的最小孔
- III. **最差适应(worst fit)**: 分配最大的孔

## 碎片

第8章 内存管理

48

### 1. 外部碎片(external)

- **区外产生的碎片**。分区大小为2byte、2byte、2byte的不连续的空间, 不能分配内存请求为6byte的进程

### 2. 内部碎片(internal)

- **区内产生的碎片**。有18464byte大小的孔, 要分配内存请求为18462byte, 会剩下2byte的孔, 怎么办?

### 解决的方法(solution)

**紧缩(compact)**: 移动内存内容, 把所有的空闲空间合并成一整块

**注**: 只适用于动态重定位的时候, 不适用于静态定位, 但开销大

不连续分配:

- 帧：把物理内存划分为固定大小的块儿，称为帧。
- 页：把逻辑内存划分为固定大小的块儿，称为页。

一个页对应一个帧，当需要分配内存时，以页为单位分配内存。若程序大小为  $n$  页，则需要有  $n$  个帧来存放它，而这些帧是不必连续的内存空间(看下一页图)

1. 需要掌握空闲空间(空闲帧)信息

2. 为了把逻辑地址转变为物理地址，需要设置一个页表



39

逻辑地址分为两个部分：页号和页偏移

1. **页号( $p$ )** 是页表的索引，相应页表中存储着该页所在物理内存的基地址
2. **页偏移( $d$ )** 结合基地址形成物理地址

| 页号      | 页偏移     |
|---------|---------|
| $p$     | $d$     |
| 20 bits | 12 bits |

20位页号可指向 $2^{20}$ 个帧，如每个帧大小 4k

那么，32位逻辑地址可覆盖的内存范围是  $2^{20} \times 2^{12} = 2^{32} = 4GB$

页表部分，见PPT，稍微优点绕，跟数据库那个索引似的，考的不多，看懂PPT例子即可（大约20分钟看完）

## 虚拟内存

唯一重要的是页置换算法

- FIFO
- 最优置换
- LRU

- 近似LRU

附加引用位

二次机会

增强型二次机会

- 基于计数（考过简答）

## 存储结构

### 4 磁盘调度算法

第12章 大容量磁盘结构

23

1. FCFS 算法
2. SSTF 算法
3. SCAN 和 C-SCAN 算法
4. LOOK 和 C-LOOK 算法
5. N-STEP-SCAN 和 F-SCAN

哈尔滨工业大学（威海）计算机科学与技术学院 朴学峰

去PPT看上面这部分即可



**低级格式化或物理格式化 (Raw/Low Format)**

：将磁盘分成磁盘控制器能读与写的扇区

**逻辑格式化**：创建文件系统

**簇 (cluster)**：把若干个块组合起来形成更大的块，磁盘的 I/O 以大块为单位完成

这一页考过小知识点

我们当时10、11章不考，12章考的内容就上面这一点。在看完以上重点后有时间可以把10-12PPT简单瞄一眼。

一些杂项：

考中断流程：

当发生中断时，

- 保存当前进程的计数器 (Program Counter)
- 跳到相应的中断服务程序中，运行相应的中断服务程序

一旦中断服务程序运行结束，

- 返回到被中断的程序 (by PC)，并继续往下运行
- 或者返回到操作系统指定的程序中

考DMA特点：

DMA直接访问内存是在专门的硬件控制下，实现高速外部设备与主存储器之间自动成批交换数据，尽量减少CPU干预的I/O操作方式

1. DMA控制器在没有CPU干涉的情况下，以块儿(Block)为单位，负责在设备缓冲器与主存之间的数据传送
2. 之前是每个字节传送完成后就触发中断表示完成，而DMA是每完成块儿传送后触发中断
3. 有利于高速 I/O设备传送数据，接近于内存速度

考DMA读操作过程：

① Programmed I/O is used to prepare disk controller for transfer by providing required information and initiating transfer

② DMA transfer from disk to memory.

③ Upon completion of transfer, disk controller sends completion interrupt to CPU.

磁盘的表面可以**逻辑**划分为多个**磁道**，每个磁道再划分为**扇区**

磁盘控制器负责设备与计算机之间的逻辑交互

考多处理器:

### 多处理器系统的优点

- 增加了吞吐量 (throughput)
- 方便扩展
- 增加了可靠性, graceful degradation or fault tolerance

---

#### 1. 非对称处理器, 又称异构多处理器, Asymmetric Multiprocessor(AMP)

- 处理器在结构上不同
- 一个处理器负责运行操作系统, 其它处理器运行其他程序, 处理器之间有主从关系

#### 2. 对称处理器, 又称同构多处理器 Symmetric Multiprocessor(SMP)

- 各处理器在结构上完全相同
- 操作系统可以运行在任何一个处理器上, 处理器之间没有主从关系。

#### 3. Multi-Core - 多核处理器

- 一个处理器上有多个CPU, 也就是多个处理器集成到一个芯片上, 可以是对称多处理器, 也可以是非对称多处理器

集群优缺点:

## 优点 - 高性能

计算机集群提供了更快的处理速度，更大的存储容量，更好的数据完整性，更高的可靠性和更广泛的可用性资源

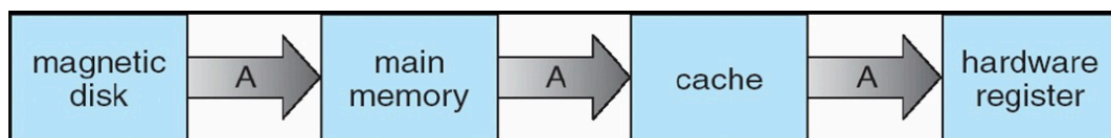
## 缺点 - 高费用

但是，需要更昂贵的实现和维护。相比一台计算机，这将导致更高的运行开销

考IO：

### 举例：把整数A 从磁盘移动到寄存器

第一章 导论



把磁盘上的  $A=1$  读到内存，然后加一，把结果存储到硬盘的流程

1. 先从磁盘上读取  $A$  到内存上
2. 复制  $A$  到高速缓存上
3. 为了加一，复制  $A$  到 CPU 的寄存器上
4. 加一后，把结果写入磁盘

被动、活动

- 进程是运行中的程序，是系统的运行单元

## 程序(program) vs 进程(process)

程序是被动实体，而进程是活动实体

1. 在一个系统中有无数个进程在同时运行，运行在一个或多个CPU上，进程之间通过复用CPU并发运行
2. 运行进程需要分配一定的资源，如CPU, 内存, I/O设备, 存储设备等。当进程结束时，也应收回已分配的资源，从而让进程有效使用这些资源

第一个PPT的第五节：操作系统的结构及操作部分。出简答，务必仔细看一遍